

BÀI TẬP LỚN | ĐẢM BẢO CHẤT LƯỢNG VÀ KIỂM THỬ PHẦN MỀM

MUTATION TESTING

TRONG ĐÁNH GIÁ CHẤT LƯỢNG BỘ TEST

Sinh viên: **Nguyễn Hoàng Việt** | MSSV: **K225480106074**

Lớp: **K58KTP.01** | GVHD: **TS. Nguyễn Tuấn Linh**

Trường Đại học Kỹ thuật Công nghiệp | Khoa Điện Tử | 2026

NỘI DUNG TRÌNH BÀY

01

Lý do chọn đề tài & Giới thiệu vấn đề

03

Mutation Score – Công thức định lượng

05

Ưu điểm, Nhược điểm & Giải pháp AI Infusion

07

Kết quả: Bộ test yếu vs. Bộ test mạnh

09

Kết luận & Định hướng phát triển

02

Cơ sở lý thuyết – Các khái niệm cốt lõi

04

Toán tử đột biến & Phân tích giá trị biên

06

Demo thực nghiệm – Bài toán is_adult

08

Phân tích dữ liệu & Ma trận so sánh

10

Tài liệu tham khảo & Kết thúc

VẤN ĐỀ VÀ LÝ DO NGHIÊN CỨU

⚠ THỰC TRẠNG

- Code Coverage 100% vẫn có thể bỏ sót lỗi nghiêm trọng.
- Bộ test "hình thức" – chạy qua dòng code nhưng không kiểm tra tính đúng.
- Test thiếu assertion → mutant sống sót → lỗi lọt lưới lên production.
- Lỗi logic tại giá trị biên không được phát hiện bởi phương pháp truyền thống.

✓ GIẢI PHÁP

MUTATION TESTING (Kiểm thử đột biến)

Chủ động chèn lỗi nhân tạo vào mã nguồn → kiểm thử lại năng lực phát hiện lỗi của chính bộ test.

CÁC KHÁI NIỆM CỐT LÕI



Chương trình gốc (P)

Mã nguồn ban đầu đã vượt qua toàn bộ test hiện có.



Biến thể đột biến (Mutant - M)

Bản sao lỗi của P, được tạo bằng cách thay đổi một lỗi cú pháp nhỏ.



Killed Mutant (Bị tiêu diệt)

Mutant bị phát hiện vì ít nhất 1 test case cho kết quả khác P.



Survived Mutant (Sống sót)

Mutant sống sót → bộ test có "điểm mù" → nguy hiểm!



Equivalent Mutant (Tương đương)

Đột biến thay đổi cú pháp nhưng ngữ nghĩa không đổi → không thể bị tiêu diệt.

MUTATION SCORE – CHỈ SỐ ĐỊNH LƯỢNG CHẤT LƯỢNG

CÔNG THỨC TÍNH MUTATION SCORE

$$MS(T) = K / (M - Me) \times 100\%$$

K

Số mutant bị tiêu diệt

M

Tổng số mutant sinh ra

Me

Số mutant tương đương

T

Bộ test suite cần đánh giá



Ngưỡng đánh giá: $MS < 70\%$ (Yếu - Nguy hiểm) | $70\% - 90\%$ (Trung bình) | $MS = 100\%$ (Mạnh - Đạt chuẩn)

TOÁN TỬ ĐỘT BIẾN & PHÂN TÍCH GIÁ TRỊ BIÊN

ROM

Relational Operator Mutation

- $\geq \rightarrow >$
- $\geq \rightarrow ==$
- $\geq \rightarrow <=$

Bị bỏ sót bởi Code Coverage

AOM

Arithmetic Operator Mutation

- $+$ $\rightarrow -$
- $\times \rightarrow \div$
- **mod** $\rightarrow \div$

Phát hiện lỗi phép tính sai

LOM

Logical Operator Mutation

- **AND** $\rightarrow$ **OR**
- **NOT** $\rightarrow$ (bỏ)
- **IF** $\rightarrow$ (!IF)

Phát hiện lỗi logic điều kiện

Mối liên hệ với Phân tích Giá trị Biên (BVA)

Khi toán tử ROM biến đổi (vd: $\geq \rightarrow >$), hành vi sai lệch CHỈ xảy ra tại giá trị biên chính xác. Mutation Testing cưỡng bách tester phải thêm test tại điểm ON/OFF để đạt Mutation Score 100%.

ƯU ĐIỂM, NHƯỢC ĐIỂM & GIẢI PHÁP AI INFUSION

✓ ƯU ĐIỂM

- Phá vỡ ảo tưởng Code Coverage
- Phát hiện Test rỗng (Missing Assertions)
- Định lượng thực chất năng lực bộ test

⚠ NHƯỢC ĐIỂM

- Bùng nổ chi phí tính toán (hàng triệu dòng code)
- Equivalent Mutant: khó tự động xử lý hoàn toàn
- Đòi hỏi hạ tầng CI/CD mạnh

🤖 GIẢI PHÁP: AI INFUSION

1. LLM sinh test case biên thông minh → tự động nhắm đúng điểm ON/OFF.
2. Self-Healing Tests: AI tự cập nhật assertion khi code thay đổi.
3. Graph Neural Network nhận diện Equivalent Mutant → loại bỏ tự động.
4. Smart Mutation: chỉ test vùng code nguy cơ cao → giảm 70-80% chi phí tính toán.

DEMO THỰC NGHIỆM – HÀM is_adult & 3 MUTANTS

Hàm Gốc (P) (✓ CHUẨN)

```
def is_adult(age):  
    if age >= 18:  
        return True  
    else:  
        return False
```

Mutant 1 (ROM: >= → >)

```
def is_adult(age):  
    if age > 18: # LỖI!  
        return True  
    else:  
        return False
```

Mutant 2 (ROM: >= → ==)

```
def is_adult(age):  
    if age == 18: # LỖI!  
        return True  
    else:  
        return False
```

Mutant 3 (Logic nhánh else)

```
def is_adult(age):  
    if age >= 18:  
        return True  
    else:  
        return True # LỖI!
```


KẾT QUẢ THỰC NGHIỆM – BỘ TEST YẾU vs. BỘ TEST MẠNH

Tiêu chí	Bộ test BAN ĐẦU (2 TC)	Bộ test AI TỐI ƯU (3 TC)	Xu hướng
Số test case	2	3	↑ +50%
Statement Coverage	100%	100%	→ không đổi
Branch Coverage	100%	100%	→ không đổi
Mutant 1 (>= → >)	 SỐNG SÓT	 TIÊU DIỆT	✓ Vá lỗi biên
Mutant 2 (>= → ==)	 TIÊU DIỆT	 TIÊU DIỆT	→ ổn định
Mutant 3 (logic else)	 TIÊU DIỆT	 TIÊU DIỆT	→ ổn định
MUTATION SCORE	66.67%	100.00%	↑ +33.33%

💡 Bài học: Code Coverage 100% vẫn để lọt lỗi nghiêm trọng! Chỉ thêm 1 test case tại age=18, Mutation Score tăng từ 66.67% → 100%.

KẾT LUẬN & ĐỊNH HƯỚNG PHÁT TRIỂN

01

Code Coverage là không đủ

MS đã chứng minh: 100% Coverage vẫn lỗi biên.
MS là thước đo thực chất hơn.

02

Mutation Testing định lượng

Tăng từ 66% → 100% chỉ với 1 test biên. Shift-Left
phát hiện lỗi sớm.

03

AI Infusion mở kỷ nguyên mới

LLM sinh test biên thông minh, tự động hóa toàn
bộ quy trình kiểm thử.

🚀 Định hướng: Smart/Selective Mutation • Self-Healing Test Framework • Tích hợp IDE Plugin

Trân trọng cảm ơn thầy/cô đã lắng nghe!

Nguyễn Hoàng Việt | K58KTP.01 | Đại học Kỹ thuật Công nghiệp Thái Nguyên